TsunaGo 繋

JAPAN × INDIA BUSINESS CONNECTIONS

Technical Report

How TsunaGo works — architecture, algorithms, AI layer, security, deployment and the questions judges ask

CONTENTS

1. The 60-second version
2. Tech stack and why
3. Repository map
4. How a request travels
5. The front end
6. The scoring engine (no AI)
7. Each stage, in depth
8. The AI layer
9. Sign-in, Gmail and security
10. Saved ventures (storage)
11. Deployment
12. Running and checking it locally
13. When something breaks
14. Honest limitations
15. Technical questions judges ask
16. Glossary

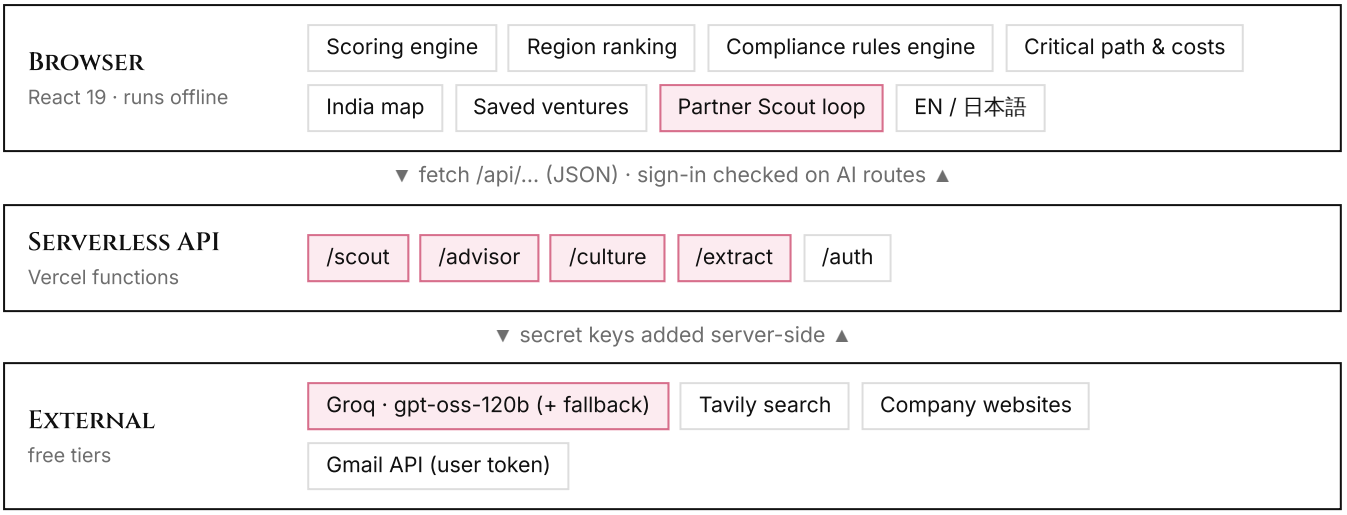
~7.9K	5	37	20
lines of application code	serverless API routes	compliance rules with sources	judge questions answered (§15)

Live demo: https://tsuna-go-shinazugawa.vercel.app *· Source: github.com/Rudhratej09/TsunaGo-Shinazugawa · docs/TECHNICAL.md*

Everything about how TsunaGo is built and why, written so you can explain any part of it to a judge, debug it on stage, or extend it after the hackathon.

1. The 60-second version

TsunaGo is a single-page web app (React) plus a handful of **serverless API functions** (Node) deployed on **Vercel**. The browser does almost everything: navigation, scoring, maps, saving your plan. The server exists for one reason: **to keep API keys secret**. Whenever the app needs an AI model or a web search, the browser calls `/api/...`, the server adds the secret key, calls Groq (the model) or Tavily (web search), and sends back the result.



Pink = uses a language model. Everything else is deterministic.

Two kinds of logic, kept deliberately separate:

DETERMINISTIC (NO AI)	AI (LANGUAGE MODEL)
Partner checks and scores, region ranking, compliance roadmap, timeline, costs, duty calculator, market ratings	Reading websites and brochures, planning searches, writing emails, advising, coaching messages
Same input → same output, explainable, instant	Flexible, but every output is checked or labelled

That split is the core design idea: **AI reads and writes; rules decide.** A judge asking "is the ranking a black box?" gets the answer "no — it's a formula you can see, the AI only fills in the facts, and every fact is checked against its source."

2. Tech stack and why

LAYER	CHOICE	WHY THIS ONE
UI framework	**React 19**	Component model, huge ecosystem, fast with Vite
Build tool	**Vite 8**	Instant dev server; also hosts the `/api` routes locally (see §4)
Animation	**framer-motion**	Page transitions, the drawer, list re-ordering
Map	**@svg-maps/india** (CC BY 4.0)	A real SVG of India's states, styled by us
Markdown	**marked** + **DOMPurify**	Render the Advisor's answers safely (DOMPurify strips scripts)
Auth tokens	**jose**	Verify Google's signed ID tokens and sign our own session cookie

LAYER	CHOICE	WHY THIS ONE
AI model	**Groq** running `openai/gpt-oss-120b` (free tier)	Fast, free, OpenAI-compatible API. Swappable (§8)
Web search	**Tavily** (free tier)	Search API built for AI agents; returns clean results with dates
Hosting	**Vercel** (free)	Static site + serverless functions from one repo, auto-deploys on push to `main`
Lint	**oxlint**	Very fast linter, catches React hook mistakes

There is **no database**. Plans are saved in the browser (`localStorage`). That keeps the prototype free and private, at the cost of not syncing across devices (§14).

3. Repository map

```
TsunaGo-Shinazugawa/
├── README.md               setup, features, demo path
├── docs/PITCH.md           3-minute script, Q&A cheat sheet, checklist
├── docs/TECHNICAL.md       this file
└── app/                    the whole application (Vercel "Root Directory")
    ├── vercel.json         Vercel settings: Vite build, API functions may run 60 s
    ├── vite.config.js      dev server + mounts /api routes locally
    ├── .env.example        list of environment variables (copy to .env.local)
    ├── public/logo.png     the logo (swap it here; see src/brand.js)
    │
    ├── api/                one tiny file per Vercel function; each just calls server/
    │   ├── scout.js  advisor.js  culture.js  extract.js  auth.js
    │
    ├── server/             server-side logic (runs on Vercel AND in the Vite dev server)
    │   ├── llm.js          provider config, model fallback, JSON calls, request helpers
    │   ├── chat.js         multi-turn chat with web-search tool calling (Advisor)
    │   ├── search.js       Tavily web search
    │   ├── scout.js        Partner Scout tools: plan, search, read, draft, digest
    │   ├── extract.js      brochure → structured partner profile (JSON schema)
    │   ├── advisor.js      Advisor: profile injection + app context
    │   ├── advisorPrompt.js the team's advisor prompt, verbatim
    │   ├── culture.js      message coach + negotiation prep
    │   └── auth.js         Google sign-in, session cookie, requireUser guard
    │
    └── src/                the browser app
        ├── main.jsx        entry point; wraps App in the sign-in provider
        ├── App.jsx         the shell: routing, state, header, tabs, drawer, page switch
        ├── auth.jsx        Google button + sign-in state (browser side)
        ├── brand.js        logo path + intro on/off
        ├── i18n-core.js    translate(): English text is the key
        ├── i18n.jsx        useI18n() hook (t, tm, lang)
        ├── i18n-ja.js      ~850 Japanese translations
        ├── index.css       all styles; design tokens at the top (:root)
        ├── components/
        │   ├── Journey.jsx     stage list, stage tabs, checklist drawer, Previous/Next bar
        │   ├── VentureMenu.jsx saved-venture switcher (header)
        │   ├── Market.jsx      stage 1 market view
        │   ├── IndiaMap.jsx    clickable SVG map with petal "presence"
        │   ├── SidePanel.jsx   analytics panel for a partner or region
        │   ├── Charts.jsx      small SVG charts used by the panel
        │   ├── Intro.jsx       opening logo + Senbonzakura petal animation
        │   └── Sakura.jsx      background petals that react to the pointer
        ├── pages/          one file per page (Setup, Options, Scout, Analyze, Advisor,
        │                   Compliance, Culture, Plan, Home, Why)
        ├── lib/
        │   ├── engine.js       scoring engine (checks, scores, regions, pilot plan)
        │   ├── compliance.js   compliance rules engine (schedule, critical path, costs)
        │   ├── scout.js        Partner Scout loop (runs in the browser)
        │   ├── ai.js           brochure extraction client + quote verification
        │   ├── gmail.js        send email / read replies with the user's Gmail
        │   ├── ventures.js     saved ventures (all browser storage goes through here)
        │   └── geo.js          map projection + state names
        └── data/
            ├── catalog.js      industries, budgets, clusters, demo partners, Japanese presence
            ├── market.js       stage 1 sourced market data
            ├── compliance.js   stage 4 rules, sources, change feed, entry-mode table
            ├── culture.js      stage 5 playbook + checklist
            ├── samples.js      two sample brochures (with cached results for offline demo)
            └── scoutDemo.js    simulated Partner Scout run with fictional companies
```

Rule of thumb when reading the code: `data/` is *what we know*, `lib/` is *how we decide*, `pages/` and `components/` are *how it looks*, `server/` is *anything that needs a secret key*.

4. How a request travels

Example: you click Coach this message in stage 5.

1. `pages/Culture.jsx` runs `fetch('/api/culture?action=coach', { method: 'POST', body: JSON })`.
2. **On Vercel**, the URL `/api/culture` maps to the file `app/api/culture.js` automatically. **Locally**, `vite.config.js` mounts the same handler on the dev server. Both call `handleCulture()` in `server/culture.js`, so local and production behave identically.
3. `handleCulture` first calls `requireUser()` (§9): if sign-in is configured and you're not signed in, it answers `401 signin_required`, and the page shows a Google button.
4. It reads the body, cleans the input (strips `<` `>`, caps the length), builds a prompt and a **JSON schema** describing exactly what the answer must look like.
5. `chatJSON()` in `server/llm.js` sends it to Groq with the secret key and parses the JSON. If Groq says the model doesn't exist, it retries with the next model automatically (§8).
6. The JSON goes back to the browser, which renders it and saves it in the current venture.

Every AI route follows this same pattern: **guard → validate → prompt + schema → model → JSON**.

Errors are standardised by `handle()` in `llm.js`: the browser always receives `{ error: 'code', message: 'human text' }` with a sensible HTTP status (400 bad input, 401 sign-in, 429 rate limit, 503 no key, 502 provider error), so pages can react (for example, show a sign-in button on `signin_required`).

5. The front end

5.1 The shell (`App.jsx`)

- **Routing** uses the URL hash: `#/market`, `#/compliance` … `readRoute()` reads it, `go(route)` changes it. Hash routing needs no server configuration and back/forward just works.
- **State** that many pages share lives here: `answers` (the SME's profile and requirements), `shortlist`, `customPartners` (from Scout/Analyze), `visited` (for the checklist). It is saved to the current venture whenever it changes.
- `result = evaluate(answers, customPartners)` runs the scoring engine on every change. It's cheap (milliseconds), so there's no caching to go stale.
- **Layout:** header (logo, venture menu, petals, checklist, language, sign-in) → stage tabs → the page → Previous/Next bar → footer. The side panel and checklist drawer float above.

5.2 The journey (`components/Journey.jsx`)

One array, `JOURNEY`, defines the stages and their pages, mirroring the six parts of the JETRO problem statement. Everything navigational is generated from it:

BUILT FROM `JOURNEY`	WHAT IT DOES
`StageTabs`	The tab row and the sub-tab row
`ChecklistDrawer`	The checklist with ticks, "next", progress
`progress()`	The "3/8" count in the header
`StepNav`	Previous/Next at the bottom of each page (follows `FLOW`)

Each item has `done(state)`, a function that decides the tick, e.g. partners are done when the shortlist isn't empty. Adding a page = one new entry here. That's why tabs, checklist and Previous/Next can't disagree.

5.3 Two languages (`i18n-*.js`)

- **English text is the key.** In code you write `t('Market & regions')`. In Japanese mode, `translate()` looks that exact string up in `i18n-ja.js`; if it's missing, English shows.
- Placeholders: `t('{n} of {m} steps done', { n: 3, m: 8 })`.
- The engine produces **messages, not strings** (`msg('Weak customer access in {ind}…', {…})`), so its explanations can be rendered in whichever language the viewer picks later.
- Large bilingual data (market overviews, compliance rules, playbook) stores `{ en, ja }` side by side instead, because it's content, not interface text.
- The AI routes receive `lang` and are told which language to answer in.

5.4 Look and feel

- **Design tokens** at the top of `index.css` (`--ink`, `--sakura-deep`, fonts…). Change a colour once, it changes everywhere.
- Fonts: Cinzel (display, the JETRO-like serif), Cormorant Garamond, Inter, Noto Serif JP.
- **Intro** (`Intro.jsx`): canvas animation. The logo starts zoomed in, ~1,500–4,200 petals sweep across in an S-shaped "river", then the logo flies to its place on the home page (it measures the `[data-logo-target]` element so it lands exactly). Click to skip.
- **Background petals** (`Sakura.jsx`): a canvas behind the page; petals get pushed by the pointer. Toggle with ✿ (remembered).
- **Map** (`IndiaMap.jsx`): SVG paths for each state; "presence" is drawn as petals clipped to a state's outline; states are clickable when a page passes `onStateClick`.

---

6. The scoring engine (no AI)

File: `src/lib/engine.js`. Pure functions: same answers in, same scores out.

6.1 Service model

```
B = needs local installation AND has no own service team in India
A = everything else
```

Model B changes the rules: partners must be able to install, and technical capability counts 1.6× (that's why its dots turn pink on Requirements).

6.2 Essential checks (pass / fail, before any scoring)

CHECK	FAILS WHEN
Sells to your customers	partner's access to your industry < 55/100 (service-only firms get a *warning*)
Can install & service	model B and (no installation, or fewer than 5 engineers)
Fits your budget	partner's minimum commitment > your budget cap (₹ lakh; custom allowed)
No competing principal	partner already sells a competing brand

A partner that fails any check is **excluded, with the reason shown**, regardless of score. This is deliberate: a high score must never hide a deal-breaker.

6.3 Weighted score (0–100)

Five sub-scores, each 0–100:

```
reach     = industry access × 0.75 + coverage of your top-2 regions × 25  (−30 service-only)
technical = engineers × 5 (max 100) × 0.8 + 20 if they calibrate
cost      = 100 − (margin% + service fee%) × 1.6 − (min. commitment / budget) × 25
speed     = 100 − (response hours − 8) × 1.6
japan     = 25 + Japanese principals × 12
```

Weights are **the SME's own priority dots** (1–5), with technical × 1.6 in model B (× 0.5 in A). Score = weighted average. Sorting: clean passes → passes with a warning → exclusions.

6.4 Regions

Each cluster (Pune–Chakan, Chennai, NCR…) gets a weighted mix of demand for your industry, Japanese ecosystem, logistics, talent, cost and **how many eligible partners operate there** (weighted more in model B, because you need a partner near the customer). Cost weighs more when the budget is ≤ ₹40 lakh. Stage 1's sourced state comparison (§7.2) is separate from this index.

6.5 Open checks

Every partner fact has a status: **verified** (checked against a reference), **claimed** (the partner says so; quote found), **pending** (not yet checked). Anything not verified becomes an "open check" for the pilot plan. This is how the app turns AI output into a to-do list instead of pretending it's true.

---

7. Each stage, in depth

7.0 Start — Profile & Requirements (`pages/Setup.jsx`)

Three preset SME profiles fill sensible defaults; six questions + priority dots refine them. Custom budget (₹ lakh) and a custom industry (with a "closest sector for data", shown honestly as "Medical imaging (data: Electronics & EMS)") are supported. Everything feeds `answers`.

7.1 Market & regions (`components/Market.jsx` , `data/market.js`)

- For each of the 5 industries: an overview, demand signals, barriers, target customer — every figure with a **source link and date**.
- 3-state comparison on four factors, rated **Strong / Moderate / Limited / Unknown**:
  1. *Buyer base* — sourced (ACMA, state agencies, NITI Aayog, ASI…)
  2. *Japanese ecosystem* — computed from the Embassy of Japan/JETRO company list (≥200 Strong, ≥100 Moderate, else Limited)
  3. *Demand for your product* — **always Unknown**: no dataset knows that; it needs buyer interviews
  4. *Service practicality* — "To check" in model B, "Not required" in A
- Suggested states = buyer base × 2 + Japanese ecosystem.
- The map is clickable; states outside the comparison say so instead of guessing.

7.2 Partners — compare & shortlist (`pages/Options.jsx` , `SidePanel.jsx`)

The demo partner list (`data/catalog.js` , **fictional companies**) plus anything found by Scout or Analyze, scored by the engine. Click a partner for the side panel: checks, sub-scores, evidence with quotes, open checks, a cost scenario. ✿ adds it to the shortlist used by later stages.

7.3 Analyze a brochure (`pages/Analyze.jsx` , `server/extract.js` , `lib/ai.js`)

1. Paste brochure text → `/api/extract` .

2. The model must answer in a strict JSON schema (type, HQ state, engineers, install, margins…) and give a **verbatim quote for every fact**.
3. Back in the browser, `findQuote()` normalises both texts (case, quotes, dashes, spaces) and looks for each quote. Found → `claimed`; not found → `pending` (flagged as unverified).
4. The profile is converted into the partner shape and scored like any other partner. Offline safety net: the two sample brochures have cached results, so the demo works without keys.

7.4 Partner Scout — the AI agent (`lib/scout.js` + `server/scout.js`)

Why the loop runs in the browser: Vercel's free plan stops a function after ~60 s, and an agent run takes longer. So the server exposes short **tools**, and the browser is the orchestrator, calling them one by one and drawing each step live.

TOOL (`/API/SCOUT?ACTION=`)	WHAT IT DOES
`status`	Which model/search are configured (no sign-in needed)
`plan`	Turns the goal into requirements + 2–3 search queries (JSON schema)
`search`	Runs the queries on Tavily; **drops directories and marketplaces** (IndiaMART, Justdial, LinkedIn, Amazon… ~30 domains)
`read`	Fetches one company website (+ its contact page), converts HTML to text, asks the model for a profile with quotes and contact details
`draft`	Writes a bilingual (EN/JA) introduction email
`digest`	Summarises a partner's reply: intent, EN/JA summary, next step

The loop (`runScout`):

```
round 1: plan → search → read sites (2 at a time) → score with the engine
         if ≥ 2 partners pass all checks → done
round 2 (reflect): tell the planner what failed and which sites were already read
         → new queries → search → read → score
finally: draft an email to the best eligible partner
```

Verification (`readToPartner`): every quote is matched against the page text; an email or phone counts as verified **only if it literally appears on the company's own page** (phone digits must match). Unverified details are shown as unverified. **Nothing is ever sent automatically.**

Simulated / replay: `data/scoutDemo.js` is a full run with fictional `.example` companies, and **Save run / Replay saved run** records and replays a real run — your stage backup when Wi-Fi fails.

7.5 Sending email and reading replies (`lib/gmail.js`)

- Uses Google's token client in the browser with the user's consent: scope `gmail.send` to send, `gmail.readonly` to read replies. **Tokens stay in browser memory**, never on our server.
- Builds a standard RFC 2822 email (UTF-8, Japanese-safe subject encoding), base64url-encodes it, calls the Gmail API. A confirm step is mandatory; only verified addresses; never to `.example`.
- "Check replies" reads **only the thread it started**, strips quoted text, and sends the reply to `digest` for a bilingual summary.

7.6 Localize & price — the Advisor (`pages/Advisor.jsx`, `server/advisor.js`, `server/chat.js`)

- The system prompt is the team's advisor prompt **verbatim** (`advisorPrompt.js`). The SME profile form is injected where the prompt says `{{SME_PROFILE}}` (missing fields → "not provided"), followed by an

`<app_context>` block: today's date, language, "Partner Finder = Partner Scout", whether web search is available.

- **Tool calling:** the model is offered a `web_search` function. When it calls it, the server runs Tavily and feeds results back (up to 4 rounds, ~30 s budget, then it must answer). Sources are returned to the page with dates.
- The answer is Markdown → `marked` → `DOMPurify` (scripts, images, forms removed) → evidence tags `[F] [U] [E] [H]` turned into coloured chips.
- The last 30 messages are sent each turn; the conversation is saved in the venture.

7.7 Compliance (`pages/Compliance.jsx` , `lib/compliance.js` , `data/compliance.js`)

- **Rules as data.** Each step: `when(input)` (does it apply?), phase, dependencies, authority, portal, documents, days `[min,max]` , government and professional fees `[min,max]` in ₹, owner, risk if missed, source, and **verification status**: `official` · `secondary` (sources agree) · `conflict` (sources disagree — verify) · `general` .
- **Engine:** filter steps for your wizard answers → schedule each one after its dependencies (min / likely / max) → walk back from the last step to find the **critical path** → sum one-time vs yearly costs.
- Your stage-2 partner can be selected: it becomes the owner of partner-run steps (IEC, GST, customs) and sets the main state.
- Also: entry-mode comparison, a duty calculator (`BCD + 10% surcharge on BCD + IGST on the total` , CEPA rate optional), a change feed of 2025–26 rule changes, CSV export, print/PDF, optional professional sign-off.

7.8 Culture & negotiation (`pages/Culture.jsx` , `server/culture.js` , `data/culture.js`)

- **Playbook:** 10 Japan-vs-India norms, each ending in an action; framed as common practice.
- **Message coach** (`action=coach`): returns `rewritten` (English), `back_translation` (Japanese), `changes` , `risks` (how the original could be misread), `tone_note` .
- **Negotiation prep** (`action=negotiate`): likely asks + responses, give-get concession ladder, red lines, bilingual phrases, prep list. Rules in the prompt: no stereotypes, no invented facts.

7.9 Pilot & operations (`pages/Plan.jsx`)

Builds a pilot timeline (weeks from your launch timing) with milestones, owners and the open checks from your shortlisted partners, plus a printable brief.

---

8. The AI layer

File: `server/llm.js` . **Provider-agnostic**: the rest of the app never talks to Groq directly.

PROVIDER	VARIABLE	DEFAULT MODEL
Groq (default)	`GROQ_API_KEY`	`openai/gpt-oss-120b` → fallback `openai/gpt-oss-20b` → `qwen/qwen3.6-27b`
Hugging Face router	`HF_TOKEN`	Qwen2.5-72B
OpenRouter	`OPENROUTER_API_KEY`	Llama 3.3 70B (free)
Ollama (offline)	—	qwen2.5:7b on your laptop
Anthropic	`ANTHROPIC_API_KEY`	Claude (uses its own web search tool in the Advisor)

The first provider with a key wins, unless `LLM_PROVIDER` is set; `LLM_MODEL` overrides the model.

Key mechanisms:

- **`postChat()` model fallback.** Providers retire free models with little notice (Groq retired Llama 3.3 70B on 16 Aug 2026 — that was the "404"). If the API answers 404, or 400 saying the model is gone, the next model in the list is tried automatically.
- **Reasoning models.** gpt-oss "thinks" before answering, and that thinking counts against the token limit; so `reasoning_effort: 'low'` is sent and ~2,000 tokens of headroom added.
- **Structured output.** `chatJSON()` asks for JSON matching a schema and parses it, even if the model wraps it in prose (it extracts the outermost `{…}`).
- **Readable errors.** `providerError()` returns the provider's own message and the model tried, e.g. "groq API error 404 (model …): The model does not exist".

Prompt-safety habits used everywhere: user text is clipped to a max length and stripped of `< >` before going into prompts; page/brochure text is wrapped in `<page>…</page>` so the model treats it as data; schemas restrict outputs to known fields and enums.

---

9. Sign-in, Gmail and security

9.1 Google sign-in (`server/auth.js`, `src/auth.jsx`)

```
Browser: Google button ──► Google returns a signed ID token (JWT)
Browser ──POST /api/auth?action=google {credential}──► Server
Server: verify signature against Google's public keys (jose), check audience = our client ID,
        email verified, optional ALLOWED_EMAILS allow-list
Server ──Set-Cookie: tsunago_session (HttpOnly, SameSite=Lax, Secure, 7 days)──► Browser
```

- No Google *client secret* is needed (ID-token flow).
- `requireUser()` guards every AI route: refuses cross-site POSTs (Origin check) and requests without a valid session. If `GOOGLE_CLIENT_ID` isn't set (local dev), routes are open.
- **Why "origin_mismatch" happened:** Google only accepts origins listed in Cloud Console. Vercel's per-deployment URLs (`…-ng3pf1r31.vercel.app`) change every deploy, so on any non-production host the app now shows "Sign in on the main site →" instead of the button (`VERCEL_PROJECT_PRODUCTION_URL` tells it the production host).

9.2 What protects what

RISK	PROTECTION
Leaked API keys	Keys only in server env vars; browser never sees them
Strangers burning our free quota	Sign-in required on AI routes; optional email allow-list
Cross-site request forgery	Origin check + SameSite cookie
Scout used to reach private networks (SSRF)	`safeURL()` allows only public http(s) hostnames (no IPs, localhost, `.internal`); redirects followed by hand and re-checked each hop; 10 s timeout; 400 KB cap
Malicious HTML in AI answers	DOMPurify sanitising; links open in new tab with `rel=noreferrer`
Prompt injection from websites	Page text marked as data; strict schemas; every claim verified against the page anyway

RISK	PROTECTION
Emails sent by mistake	Explicit confirm; verified addresses only; never automatic
Scraping directories / personal data	Directory domains skipped by policy; only companies' own published contacts

10. Saved ventures (storage)

File: `src/lib/ventures.js`. **Every** read/write of plan data goes through `vget(key)` / `vset(key, value)`, which prefix the key with the active venture:

```
tsunago-ventures                    → { active: "k3x9…", list: [{id, name, created, updated}] }
tsunago-v:k3x9…:answers             → the SME's answers
tsunago-v:k3x9…:shortlist           → ["coromandel", …]
tsunago-v:k3x9…:advisor-chat        → Advisor conversation
… 13 keys per venture (see VENTURE_KEYS)
tsunago-lang, tsunago-petals        → app-wide preferences (not per venture)
```

- **Switching** saves the registry and reloads the page, so every screen reads the new venture cleanly (simpler and safer than updating dozens of components live).
- **Duplicate** copies all keys to a new id → independent "what-if" copy.
- **Export** writes `{ format: 'tsunago-venture', version: 1, name, data }` as `.tsunago.json`; **import** validates the format and version before creating a new venture.
- **Migration:** data saved before ventures existed is moved into "My first venture" on first load.
- All storage calls are wrapped in try/catch: in private mode the app still works, it just forgets.

11. Deployment

- Vercel project **Root Directory = `app`**. Every push to `main` triggers a production deploy; every branch push gets a *preview* URL.
- `vercel.json`: framework `vite`, output `dist`, API functions may run 60 s.
- **Environment variables** (Vercel → Settings → Environment Variables):

VARIABLE	NEEDED FOR
`GROQ_API_KEY`	All AI features
`TAVILY_API_KEY`	Partner Scout search, Advisor web search
`GOOGLE_CLIENT_ID`	Google sign-in (turns route protection on)
`SESSION_SECRET`	Signing the session cookie (random 32+ chars)
`ALLOWED_EMAILS`	Optional allow-list

- **Google Cloud:** OAuth client (Web application) → *Authorised JavaScript origins* = `http://localhost:5173` and your production `https://…vercel.app`. No redirect URIs needed.
- **Deployment Protection** (Vercel) covers per-deployment URLs; share the production domain.

12. Running and checking it locally

```
cd app
npm install
cp .env.example .env.local     # paste GROQ_API_KEY and TAVILY_API_KEY
npm run dev                    # http://localhost:5173  (the /api routes work too)
npx oxlint src server api      # lint
npm run build                  # production build into dist/
```

Without keys: the simulated Scout run, the saved-run replay and the two sample brochures still work; everything deterministic (scores, market, compliance, playbook) works fully.

How changes were verified during the build: headless-browser scripts (Playwright) clicking through each flow with the AI endpoints stubbed, plus Node scripts calling the server functions with a fake provider to test fallbacks, validation and prompt assembly.

13. When something breaks

Open the browser DevTools (F12) → **Console** for errors, **Network** to see each `/api` call, its status and response. Most problems identify themselves there.

SYMPTOM	LIKELY CAUSE	FIX
"groq API error 404 …"	Model retired	Fallback should handle it; else set `LLM_MODEL` on Vercel
"rate limit reached" (429)	Free-tier limits	Wait 30–60 s; use **Replay saved run** on stage
"Not configured — needs GROQ_API_KEY"	Env var missing on that deployment	Add on Vercel, then **Redeploy**
Google "origin_mismatch"	Not on the production domain, or origin not listed	Open the production URL; add it under Authorised JavaScript origins
"Sign in with Google to run the live AI"	Session missing/expired	Sign in again (sessions last 7 days)
Friend sees a Vercel login page	Deployment Protection on preview URL	Share the production domain
Scout: "site could not be loaded"	Site blocks bots, slow, or not HTML	Normal; the agent skips it and moves on
Old version after deploy	Browser cache	Ctrl+Shift+R
Plan "disappeared"	Different browser/profile, or another venture is active	Check the venture menu; import your exported file
Advisor answer cut off	Long answer hit the token limit	Ask for one section at a time

14. Honest limitations

Say these before a judge finds them — it builds trust.

- **Demo partners are fictional.** The built-in partner list is illustrative; real partners come from Partner Scout (live web) or brochure analysis.

- **Some compliance rules are not yet verified** against primary sources — they are labelled `conflict` or `general` in the app. A CA/lawyer should confirm before acting.
- **Market data is thin for some sectors** (e.g. food processing state figures) → shown as Unknown.
- **Region index values in the partner view are illustrative**; the sourced comparison is in Market.
- **Storage is per browser.** No cloud sync yet; export/import bridges devices.
- **Free tiers** limit how many AI calls per minute/day; a production version would use paid quotas.
- **AI can still be wrong** — which is exactly why every claim is tagged, quoted or verified.

Roadmap answers: cloud-synced ventures (database + sign-in), live company-registry checks (GST/MCA), a JETRO-curated adviser directory, more sectors and states with sourced data, and a monitoring layer after entry (deadline alerts for compliance steps).

---

15. Technical questions judges ask

Answer in one or two sentences, then stop. If you don't know, say what you would measure.

Architecture and AI

Is this just a ChatGPT wrapper? No. Scoring, regions, compliance scheduling and costs are deterministic code. The AI is used for reading and writing, and its output is verified (quotes, contacts) or tagged (evidence levels). Partner Scout is an agent with tools and a reflect step, not a single prompt.

What makes Partner Scout an "agent" rather than a chatbot? It plans its own search queries, uses tools (search, read a website), observes the results, scores them with the engine, and if fewer than two partners pass it reflects on why and searches again — then it drafts outreach. The loop decides its next step from what it found.

Why does the agent loop run in the browser? Vercel's free plan stops a function after about 60 seconds, and a full run takes longer. Each tool call is short; the browser orchestrates them and shows each step live. As a bonus, the user sees exactly what the agent is doing.

Why Groq and gpt-oss instead of GPT-4 or Claude? Free, fast and open-weight, which keeps cost near zero for SMEs. The layer is provider-agnostic: one environment variable switches to Claude, Hugging Face, OpenRouter, or a fully offline Ollama model.

What happens if Groq goes down or retires the model? Automatic fallback to the next model (this already happened: Groq retired Llama 3.3 in Aug 2026 and the app moved to gpt-oss with a fallback chain). For the stage there is also a recorded run that replays offline.

What was the hardest technical problem? Three, all solved: the 60-second serverless limit (browser-orchestrated agent), model retirement (automatic fallback), and Google sign-in failing on per-deployment URLs (origin check that redirects to the production site).

Trust and accuracy

How do you stop hallucinations? Three layers: schemas restrict what the model can return; every fact needs a verbatim quote that we search for in the source text; contacts must literally appear on the company's site. Anything unverified is visibly marked.

How accurate is the extraction? We have not benchmarked it yet, and we say so. The quote check makes errors visible rather than silent. Next step: 50 hand-labelled company pages, measuring field accuracy and quote-match rate.

What stops a malicious website from manipulating the AI (prompt injection)? Page text is wrapped and marked as data, outputs must fit a strict schema, and every claim is then verified against the page by code — an injected instruction cannot make an unverified claim look verified. Nothing is ever sent without the user's confirmation.

Where does the market and regulatory data come from, and how do you keep it current? Every figure links to its source and date (ACMA, state agencies, NITI Aayog, Embassy of Japan/JETRO, RBI, DPIIT, CBIC...). Rules carry a verification level and a "checked" date; recent changes appear in a change feed. In production, a reviewer would re-check rules on a schedule.

Why do some rules say "sources conflict"? Because 2026 changed several of them (land-border FDI in March, the Income-tax Act in April, labour rules in May, the BIS machinery order). We show the uncertainty instead of picking one source.

Why don't you use IndiaMART or Justdial? Their terms forbid scraping and listings mix in personal contact data. Partner Scout skips about 30 directory and marketplace domains and reads only companies' own public websites.

Security and privacy

Is it secure? Keys are server-side only, Google sign-in signatures are verified, the session is an HttpOnly cookie, cross-site requests are refused, the website reader blocks private addresses (SSRF), AI output is sanitised before display, and emails are never sent automatically.

What data do you store? The SME's plan stays in its own browser (no database). Gmail tokens stay in browser memory. Our server stores nothing; it only forwards AI requests with the key. A production version holding SME data would need a DPDP Act (India) and APPI (Japan) review.

Can a team share a plan? Yes, by exporting a venture as a `.tsunago.json` file and importing it elsewhere. Cloud sync with sign-in is on the roadmap.

Product and scale

How does it scale to thousands of SMEs? Static front end on a CDN, stateless serverless functions, a handful of AI calls per task. Cost scales only with AI usage; moving to paid model tiers lifts the free-tier rate limits.

Can it run without internet or keys? The deterministic parts (scores, market, compliance, playbook) work fully. The simulated Scout run, recorded-run replay and two sample brochures work without keys; Ollama can run the model locally.

How is the Japanese quality ensured? About 860 interface strings are translated, content data is bilingual, and AI outputs are requested in the viewer's language. The message coach adds a Japanese back-translation so the SME can check the meaning of what it sends.

How would JETRO use this? As a first-pass tool before an SME meets an adviser: the SME arrives with a structured plan, a shortlist and open questions, so adviser time goes on judgement rather than research.

What would you build next? An extraction benchmark, primary-source checks on the remaining rules with a CA, live GST/MCA registry checks for partners, cloud-synced ventures, and deadline alerts after entry.

16. Glossary

TERM	MEANING
SPA	Single-page app: one HTML page, React swaps views without reloading
Serverless function	Code that runs on demand on Vercel for each request (`app/api/*.js`)
LLM	Large language model (here: gpt-oss-120b on Groq)
JSON schema	A description of the exact JSON shape we require from the model
Tool calling	The model asks the server to run a function (e.g. `web_search`) and continues with the result

TERM	MEANING
Agent	An LLM that plans, uses tools, observes results and decides next steps in a loop
ID token / JWT	A signed token proving who you are; we verify Google's signature
HttpOnly cookie	A cookie JavaScript can't read — protects the session from scripts
CSRF	Cross-site request forgery; blocked by origin checks and SameSite cookies
SSRF	Server-side request forgery; tricking a server into fetching internal addresses
localStorage	Small key-value storage in the browser, per site
Critical path	The longest chain of dependent steps; it decides the total time
Service model A / B	A: SME supports customers itself. B: needs a local partner to install and service
Evidence tags	[F] verified fact · [U] user input · [E] estimate · [H] hypothesis